GAMEHUB /LEARN — FIELD NOTES

GameHub

A Futuristic Gaming Platform Built with AWS & DevOps

A build log across every phase of the project — what was built, why that choice beat the alternative, the bugs found, and the concepts worth understanding cold.

Generated from `README.md` and `docs/*.md` by `scripts/build-learn.js`.
Live: `https://d2qf9zxehny6bk.cloudfront.net`

Contents

OVERVIEW	GameHub
STEP 0	Step 0 — Reproducible Local Foundation ✅ (2026-10-18)
STEP 1	Step 1 — Automated Tests + GitHub Actions CI ✅ (2026-10-18)
STEP 2	Step 2 — Game-y UI: Live Background + Self-Hosted Arcade Fonts ✅ (2026-10-18)
STEP 3	Step 3 — Real Playable Games ✅ (2026-10-19)
STEP 4	Step 4 — Real Auth + Live on AWS ✅ (2026-10-19)
STEP 5	Step 5 — Observability ✅ (2026-10-19)
STEP 8	Step 8 — /learn Documentation Site + PDF Export ✅ (2026-10-19)
DOC	Runbook — GameHub (prod)
DOC	Service level objectives — GameHub

🎮 GameHub

A Futuristic Gaming Platform Built with AWS & DevOps

`CI` `License: MIT` `Tests` `Coverage`

GameHub is a portfolio project that combines a futuristic gaming interface with a containerized Node.js backend and a MySQL database — all runnable locally with one command. It demonstrates how frontend applications, backend APIs, container networking, reverse proxies, databases, tests and CI pipelines work together.

The project is being evolved into a production-style application: automated CI/CD, infrastructure as code on AWS, monitoring, and Kubernetes (local kind cluster + Argo CD).

> **Status — Phase 4 complete (2026-10-19):** tested, CI'd, game-y, playable, and **live on AWS**. 64 automated tests, a Cognito sign-in gate, the static site on S3 + CloudFront, the API on EC2, all described as CloudFormation and deployed by keyless GitHub OIDC.

Live demo: https://d2qf9zxehny6bk.cloudfront.net — create an account (email confirmation), sign in, and play. The landing page is public; the catalogue and the games require a signed-in player.

📌 Project Status

Current phase: Phase 0 — reproducible local foundation ✅

Status: Full stack runs locally end-to-end: `make bootstrap` → seeded database, healthy containers, passing smoke tests. No cloud resources required.

✅ Completed (Phase 0)

- Futuristic gaming homepage
- Dedicated Games page
- Dynamic game catalogue loaded through a backend API (12 seeded records)
- Game category filters
- Frontend built with HTML, CSS, and JavaScript
- Node.js and Express backend
- **Local MySQL 8.4 container** with schema + seed (`db/init.sql`) — reproducible on any machine, no cloud account needed
- **Connection pool** (not a connection-per-request), **liveness/readiness split** (`/api/health` vs `/api/ready`), structured JSON logs, CORS locked to explicit origins, generic client-facing errors
- Dockerized frontend and backend with **pinned base images**, `npm ci` from lockfile, **0 npm vulnerabilities**, backend running as **non-root**
- Docker Compose orchestration with **healthchecks + ordered startup**
- Nginx reverse proxy (gzip, security headers) — backend port **not published** to the host
- `make bootstrap` one-command workflow + **smoke test** (`scripts/smoke.sh`)
- Git and GitHub repository setup

✅ Completed (Phase 1 — tests & CI)

- **14 automated tests** using Node's built-in `node:test` (zero extra test deps): unit suite (runs anywhere, no DB) + integration suite against real MySQL (skips loudly if no database is reachable)
- Behavioural coverage of the paths that matter: liveness survives a database outage while readiness returns 503, CORS refuses unknown origins, driver errors never leak to clients, malformed JSON → 400
- **Coverage gate**: ≥80% lines / branches / functions or CI fails
- **ESLint** (flat config) as a build gate
- **GitHub Actions CI — 3 jobs on every push/PR:**
  1. lint + tests + coverage gate against a MySQL **service container**
  2. **full-stack smoke test** — builds the real Docker images, runs the real compose stack on the runner
  3. **security scans** — gitleaks (full history), Trivy (CRITICAL/HIGH), npm audit
- Local equivalents: `make test` · `make lint` · `make coverage`

✅ Completed (Phase 2 — game-y UI)

- **Live animated background**: canvas starfield (3 parallax layers, pixel stars), rising neon pixel particles, rare shooting stars — plus a scrolling grid and a drifting neon orb. Zero JS dependencies, ~118 draw calls/frame
- **GDPR-clean fonts**: Press Start 2P (arcade accents) + Chakra Petch (headings/body) **self-hosted** as woff2 with OFL licenses in-repo — the Google Fonts CDN import is gone, so **zero third-party requests** at load
- **Accessibility**: `prefers-reduced-motion` visitors get a fully static page (canvas is never even created)
- Arcade styling on all interactive elements (PS2P buttons with press feel)

✅ Completed (Phase 3 — real playable games)

- **All 12 catalogue games are actually playable** — `play.html?id=<n>` fetches the title and runs one of **6 dependency-free canvas engines**: snake, breakout, survival-racing (dodge), memory, reflex targeting, pong
- Runner state machine: READY → PLAYING → GAME OVER with **Play again**, HUD (score / best / status), per-title **high scores in localStorage** (guarded), controls help, Escape → catalogue
- `GET /api/games/:id` endpoint: honest 404s for unknown *and* non-numeric ids; malformed/oversized JSON now 400/413 instead of 500
- **Frontend tests (21)**: the six engines execute in Node against a stubbed canvas — frame-driven, asserting real terminal paths; plus a **coverage contract test** that fails CI if any seeded title lacks an engine
- An engine test found and fixed a real stall bug (dead-centre breakout hit → vertical ball → unwinnable game)

✅ Completed (Phase 4 — real auth + live on AWS)

- **Cognito sign-in gate**: sign-up with email confirmation, sign-in, sign-out. The static frontend talks to Cognito's REST API directly (no Amplify, no SDK) and sends the ID token as a `Bearer` header on every `/api/*` call
- **Server-side token verification** (`backend/auth.js`): RS256 signature checked against the pool's JWKS with `node:crypto` , plus `iss` , `aud` / `client_id` , `exp` and `token_use` — no `jsonwebtoken` dependency
- **Honest fail-open**: the gate is enforced only when the pool is configured; local development runs open and the backend logs an `auth_disabled` warning at boot — never a silent bypass
- **Static site on S3 + CloudFront**: private bucket, CloudFront Origin Access Control, `/api/*` behaviour with caching disabled, HTML short-cache vs assets long-cache

- **API host on EC2** running the *same* `docker-compose.yml` as local development (nginx + Express + MySQL), with a 2 GB swap file for headroom
- **No SSH**: the security group opens port 8080 to the AWS-managed CloudFront origin-facing prefix list only; administration is via SSM Session Manager, IMDSv2 is required and the root volume is encrypted
- **Four CloudFormation stacks** (`infra/cognito.yaml`, `site.yaml`, `backend.yaml`, `github-oidc.yaml`) — one command per stack, outputs wired into the next
- **Keyless deploys**: GitHub Actions assumes a scoped role through OIDC (no AWS access keys anywhere); the workflow publishes the site, invalidates CloudFront and rolls out the API host over SSM
- **Landing page stays public** while the catalogue and the runner require a signed-in player — found by actually signing out live and following the redirect, not by reading the code
- **Versioned asset URLs** (`script.js?v=<deploy sha>`): assets are cached for a week, so a CloudFront invalidation alone still leaves returning browsers on the previous script. HTML (one-minute cache) points at the new version — found by watching a stale `script.js` run in the browser after a deploy

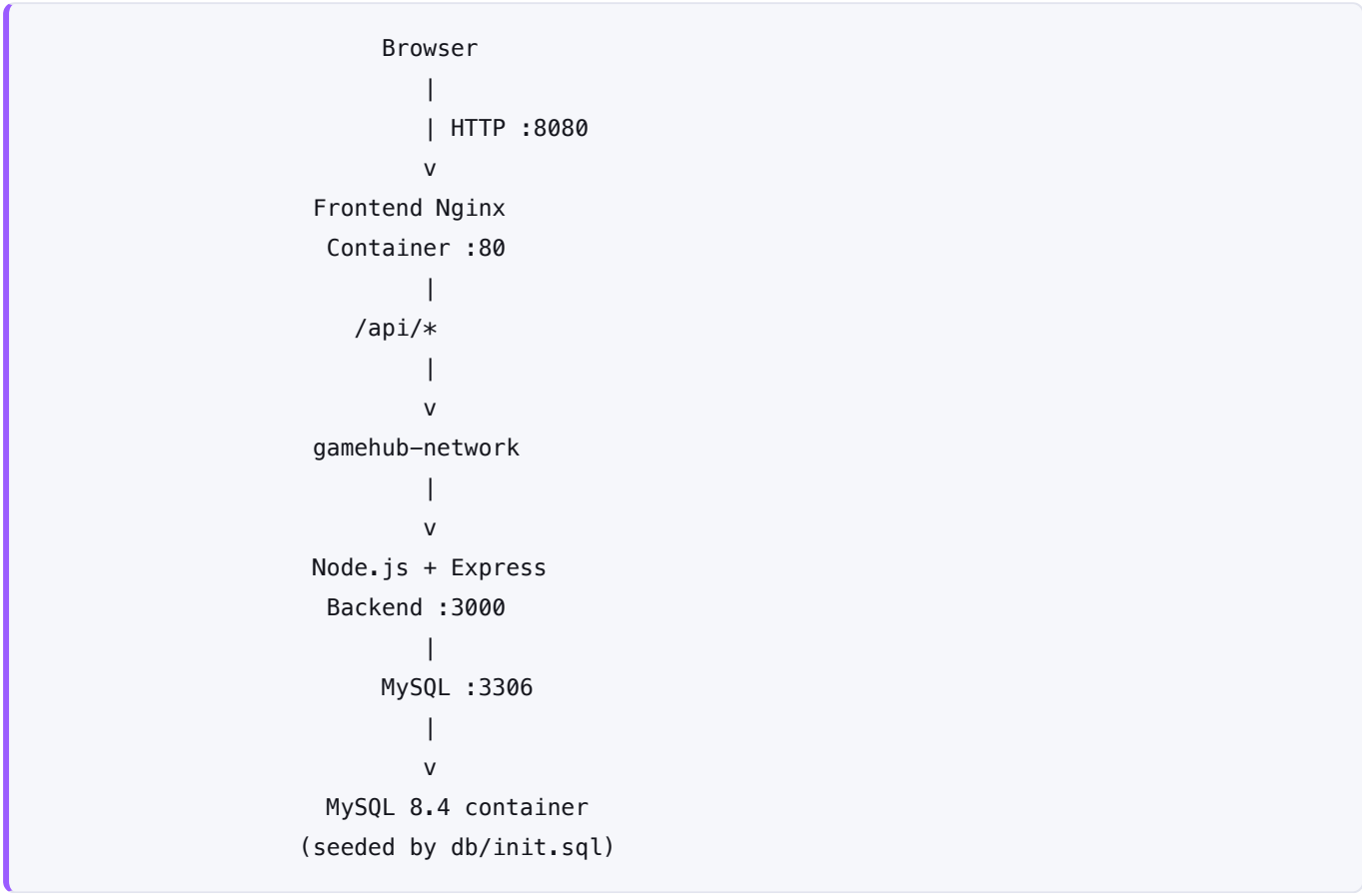
✅ Completed (Phase 5 — observability)

- **Structured request logs**: nginx emits JSON access logs (`status`, `uri`, `request_time`, `upstream_time`) — CloudWatch parses them without regex guesswork
- **CloudWatch agent on the host**: ships those logs plus container stdout to `/gamehub/prod/nginx-access` and `/gamehub/prod/app` (7-day retention set by the template), and host `mem` / `disk` / `swap` metrics into the `CWAgent` namespace — installed by user-data, no manual setup
- **Metrics derived from logs** (`AWS::Logs::MetricFilter`): request count, 5xx count, and API latency as a *numeric* extraction (`$.request_time`) for p50/p99 — nginx already knew the numbers, so the app is not instrumented with an extra library to repeat them
- **External availability check**: EventBridge runs a small Lambda every 5 minutes against the public CloudFront URL and `/api/health`; its log line becomes the `SyntheticSuccess` metric — a synthetic check without the CloudWatch Synthetics price tag
- **Eight alarms** into one SNS topic: synthetic availability (`TreatMissingData: breaching` — silence is a failure), 5xx count, API p99 latency, instance status, CPU, CPU credits, memory and disk. Host-metric alarms treat missing data as `notBreaching`, because a stopped agent is not an outage
- **One dashboard** (`gamehub-prod`): availability, requests vs errors, latency percentiles, host pressure, and two live log tables (recent 5xx, backend tail)
- **SLOs with error budgets** (`docs/slo.md`): 99.5 % availability and p99 ≤ 1.5 s over a rolling 30 days, with the measurement method for each — and an explicit list of what is **not** claimed, because a single-AZ t3.micro cannot honestly promise it
- **Runbook** (`docs/runbook.md`): one section per alarm with the exact SSM/Logs-Insights commands to confirm the cause, plus the revert-to-roll-back path
- **Verified live, not assumed**: the agent's metric dimensions were read back from `list-metrics` and then written into the template — an alarm with the wrong dimension set sits in `INSUFFICIENT_DATA` forever

🚧 Planned

- **Phase 6 — local Kubernetes (kind) + Argo CD GitOps**
- **Phase 7 — security & GDPR hardening** (data deletion flow, SBOM)
- **Phase 8 — /learn documentation site, PDF export, load-test report**

🏗️ Architecture

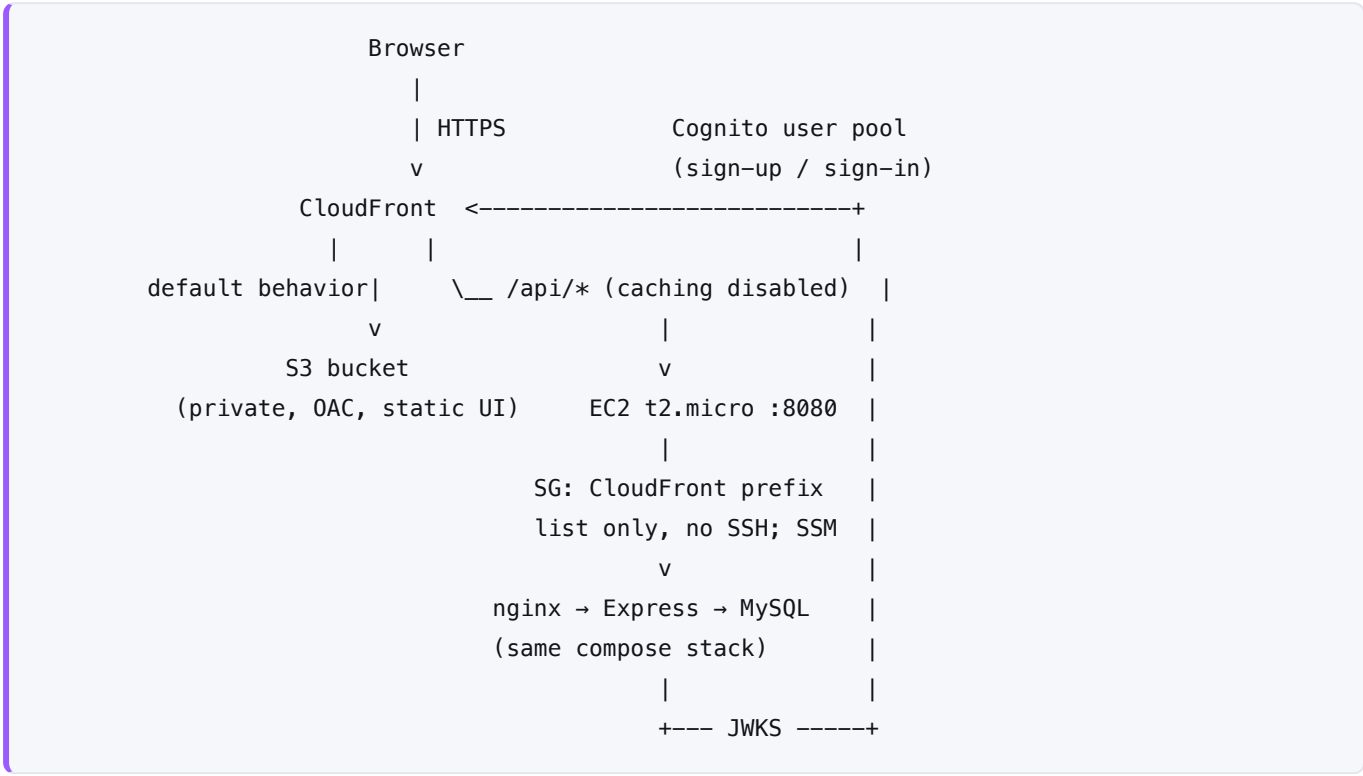


Request flow

1. A user opens GameHub in the browser.
2. Nginx serves the static frontend files.
3. JavaScript sends API requests to the frontend's `/api/` path.
4. Nginx forwards those requests to the backend container.
5. Express processes the request and queries the MySQL container over the private Docker network.
6. The backend returns JSON, which the frontend uses to render game cards.

Important: The browser accesses the frontend through port `8080`. The Nginx container communicates with the backend using Docker networking, and the backend port is **not published to the host** — the database runs as a local MySQL container (managed by hand, not by IaC — see the note in the Database section).

Live deployment (Phase 4)



The browser signs in against Cognito, receives an **ID token**, and sends it as `Authorization: Bearer <token>` on every API call. The backend verifies the token's RS256 signature against the pool's public JWKS — the frontend guard is UX, the API is the security boundary.

🧰 Technology Stack

Technology	Purpose
HTML5	Frontend structure
CSS3	Styling and responsive visual design
JavaScript	UI interactions, API requests, and category filters
Node.js	Backend JavaScript runtime
Express.js	REST API
MySQL 8.4	Relational database, local container (managed RDS via IaC in Phase 4)
Docker	Application containerization
Docker Compose	Multi-container orchestration
Nginx	Static web server and reverse proxy
Git	Version control
GitHub	Source code hosting
Colima	Local Docker runtime on macOS
Make	One-command developer workflow (`make bootstrap`)
Amazon Cognito	Player sign-up / sign-in, issues the ID token the API verifies
Amazon S3 + CloudFront	Static site hosting (private bucket, OAC) and global edge delivery
Amazon EC2	Hosts the compose stack (nginx + API + MySQL)
AWS CloudFormation	All infrastructure as code (`infra/*.yaml`)
AWS Systems Manager	Keyless administration and deploys run over SSM, without SSH
GitHub Actions OIDC	Keyless deploys: short-lived AWS credentials, no stored secrets

---

📁 Project Structure

```
GameHub/
├── frontend/
│   ├── index.html          ← landing page (hero, tournaments, leaderboard)
│   ├── games.html          ← catalogue (rendered from the API)
│   ├── play.html           ← game runner host (READY → PLAYING → GAME OVER)
│   ├── signin.html         ← Cognito sign-in
│   ├── signup.html         ← Cognito sign-up + email confirmation step
│   ├── script.js           ← homepage + catalogue logic
│   ├── play.js             ← runner: ENGINE_MAP, HUD, high scores
│   ├── engines.js          ← 6 dependency-free canvas game engines
│   ├── auth.js             ← Cognito REST client, session, page guard
│   ├── config.js           ← runtime config (local dev: auth off)
│   ├── bg.js               ← animated arcade background
│   ├── style.css
│   ├── fonts/              ← self-hosted game fonts (OFL)
│   ├── nginx.conf
│   └── test/               ← engine, mapping contract and auth tests
├── backend/
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── .env.example        ← copy to .env only for non-Docker runs
│   ├── db.js               ← connection pool
│   ├── auth.js             ← Cognito JWKS verification (no SDK)
│   ├── app.js              ← Express app (exported for tests)
│   ├── server.js           ← HTTP entrypoint
│   ├── test/               ← unit, integration and auth suites
│   ├── package.json
│   └── package-lock.json
├── db/
│   └── init.sql            ← schema + seed (runs on first boot)
├── infra/
│   ├── cognito.yaml        ← user pool + app client
│   ├── site.yaml           ← S3 + CloudFront (+ /api/* origin)
│   ├── backend.yaml        ← EC2 host, security group, IAM, elastic IP
│   ├── observability.yaml  ← log groups, metric filters, canary, alarms
│   └── github-oidc.yaml    ← OIDC provider + scoped deploy role
├── scripts/
│   ├── smoke.sh            ← end-to-end checks
│   ├── build-frontend.sh   ← builds the deployable static artifact
│   └── deploy-frontend.sh  ← S3 sync + CloudFront invalidation
├── .github/workflows/
│   ├── ci.yml              ← lint, tests, coverage, compose smoke, scans
│   └── deploy.yml          ← OIDC deploy (site + API host)
├── docs/                   ← learning notes, SLOs, runbook, ADRs
├── Dockerfile              ← frontend image (nginx)
├── docker-compose.yml      ← frontend + backend + db
├── Makefile                ← make bootstrap / up / smoke / test / db-reset
```

```
├── .dockerignore
├── .gitignore
└── README.md
```

Security note: no credentials are needed for local development — the defaults in `docker-compose.yml` are for the private Docker network only. If you run the server outside Docker, copy `backend/.env.example` to `backend/.env`. Real `.env` files are excluded from Git **and** from image builds (`backend/.dockerignore`).

🚀 Run GameHub Locally

Prerequisites

- macOS or Linux
- Git
- A Docker runtime + Docker Compose (on this Mac: Colima — start it with `colima start`)

No configuration needed. The stack ships with working local defaults: a MySQL container is seeded automatically from `db/init.sql` on first start.

1. Clone and bootstrap

```
git clone https://github.com/cijostany6-bit/GameHub.git
cd GameHub
make bootstrap        # build → start → wait for health → run smoke tests
```

2. Open GameHub

http://localhost:8080

Useful commands

```
make ps               # container status (healthy?)
make logs             # tail all logs
make smoke            # re-run the end-to-end checks
make test             # backend unit + integration tests (needs MySQL)
make test-frontend    # game-engine, mapping-contract and auth tests (Node only)
make down             # stop, keep database data
make db-reset         # wipe + re-seed the database
make clean            # full teardown incl. images and DB volume
make help             # list all targets
```

Prefer plain Compose? `docker-compose up -d --build` then `./scripts/smoke.sh` does the same.

☁️ Deploy to AWS

Everything is CloudFormation in `infra/` — five small stacks, each one a single command. Stack outputs feed the next stack, so there is no spreadsheet of ids to copy by hand.

```
export AWS_REGION=eu-central-1

# 1. Cognito user pool + app client
aws cloudformation deploy --region "$AWS_REGION" \
  --stack-name gamehub-cognito --template-file infra/cognito.yaml \
  --parameter-overrides Stage=prod

# 2. S3 + CloudFront (deployed first without an API origin)
aws cloudformation deploy --region "$AWS_REGION" \
  --stack-name gamehub-site --template-file infra/site.yaml \
  --parameter-overrides Stage=prod

# 3. EC2 host running the compose stack (needs the CloudFront domain)
aws cloudformation deploy --region "$AWS_REGION" \
  --stack-name gamehub-backend --template-file infra/backend.yaml \
  --parameter-overrides \
    Stage=prod \
    CognitoUserPoolId=<pool id> \
    CognitoClientId=<client id> \
    SiteOrigin=<https://cloudfront domain> \
    DbPassword=<16+ chars> RootDbPassword=<16+ chars>

# 4. Re-deploy the site stack with the API origin attached, then publish
aws cloudformation deploy --region "$AWS_REGION" \
  --stack-name gamehub-site --template-file infra/site.yaml \
  --parameter-overrides Stage=prod BackendOriginDomain=<ec2 public dns>

AWS_REGION=$AWS_REGION ./scripts/deploy-frontend.sh

# 5. Observability: log groups + retention, metrics from logs, the synthetic
#    check, eight alarms and the dashboard. AlarmEmail is optional — without it
#    the SNS topic is still created and can be subscribed later (see the
#    TopicArn output), it just has no subscriber yet.
aws cloudformation deploy --region "$AWS_REGION" \
  --stack-name gamehub-observability --template-file infra/observability.yaml \
  --capabilities CAPABILITY_NAMED_IAM \
  --parameter-overrides \
    Stage=prod \
    InstanceId=<ec2 instance id> \
    SiteUrl=<https://cloudfront domain> \
    AlarmEmail=<you@example.com>
```

Deploy the observability stack **before** the host's CloudWatch agent writes its first log line: the template owns the log groups so their retention is set, and an agent-created group would make that resource fail with `AlreadyExists`.

For continuous deployment, deploy the fourth stack (`infra/github-oidc.yaml`) once, then set these GitHub repository variables and push to `main` — the deploy workflow does the rest through OIDC, with no AWS keys stored anywhere:

`AWS_REGION`, `AWS_DEPLOY_ROLE_ARN`, `SITE_BUCKET`, `DISTRIBUTION_ID`, `INSTANCE_ID`, `COGNITO_USER_POOL_ID`, `COGNITO_CLIENT_ID`.

Cost note (honest): the site (S3 + CloudFront), Cognito and the four CloudFormation stacks are inside the free tier at this traffic level. The EC2 instance is the only line item: this account is on a free plan, so AWS *rejects* instance types that are not free-tier eligible (`t2.micro` failed with "not eligible for Free Tier") — the stack therefore defaults to `t3.micro`. On a non-free-tier account that is roughly $8/month. Tear everything down with `aws cloudformation delete-stack` in reverse order.

🔌 Backend API

The following endpoints are implemented in the current application.

Method	Endpoint	Purpose
GET	`/api/health`	**Liveness** — process is up (never touches the database)
GET	`/api/ready`	**Readiness** — database reachable, safe to route traffic (503 otherwise)
GET	`/api/games`	Retrieve games from the database
GET	`/api/games/:id`	Single game (powers the runner); 404 for unknown *and* non-numeric ids
GET	`/api/me`	Echoes the verified Cognito identity — proof the token was checked

The API is reachable **only through the nginx proxy on `:8080`** — port `3000` is deliberately not published to the host (use `docker-compose exec backend …` if you need direct access from inside the network).

Auth behaviour: middleware on every `/api/*` route except the two probes. If the Cognito pool is configured (that is: in the deployment), requests without a valid ID token get `401 {"error":"Unauthorized"}`. If it is not configured (local development), requests pass through and the backend logs `auth_disabled` at boot — so the app is never broken, but the bypass is never silent either.

```
curl http://localhost:8080/api/health
curl http://localhost:8080/api/ready
curl http://localhost:8080/api/games
curl http://localhost:8080/api/games/4

# with the pool configured, protected routes need the ID token:
curl -H "Authorization: Bearer $ID_TOKEN" http://localhost:8080/api/me
```

Or run the full check suite: `make smoke`.

🗄️ Database

GameHub uses **MySQL 8.4 in a container**, seeded automatically from `db/init.sql` on the first start of an empty volume. This makes a fresh clone reproducible with no cloud account and no secrets — and the deployed stack runs the *same* container on the EC2 host, so there is exactly one database setup to reason about. Moving to RDS (multi-AZ, automated backups, managed patching) is a deliberate cost/ops trade-off left for a later phase rather than a claim made here.

Reset to a clean seeded state any time with `make db-reset`.

The current games table contains these fields:

Column	Purpose
`id`	Game identifier
`name`	Game title
`category`	Category used by the frontend filters
`description`	Short game description

Sample categories include Action, Racing, and Strategy.

The frontend retrieves game records dynamically through the Express API rather than relying entirely on hardcoded game cards.

🐋 Docker Design

Frontend container

- Uses Nginx Alpine.
- Serves HTML, CSS, and JavaScript.
- Loads the custom Nginx configuration.
- Forwards `/api/` requests to the backend service.

Backend container

- Uses Node.js Alpine.
- Installs application dependencies.
- Runs the Express server.
- Reads database configuration from environment variables.
- Connects to the MySQL container over the private Docker network (no host port published).

Docker Compose

Compose manages both services and connects them through the `gamehub-network` network.

Useful commands:

```
docker-compose ps
docker-compose restart
docker-compose up -d --build frontend
docker-compose up -d --build backend
docker-compose logs -f backend
docker-compose down
```

Rebuild a service after changing its code:

```
docker-compose up -d --build frontend
```

Or:

```
docker-compose up -d --build backend
```

🔐 Security Practices

- **No secrets required for local development** — compose defaults live in the private Docker network; nothing sensitive exists in the repo.
- `.env` is excluded from Git **and** from image builds (`backend/.dockerignore`); `backend/.env.example` documents every variable.
- The backend runs as a **non-root** user and reads all configuration from environment variables (12-factor).
- The API port is **not published** to the host — only nginx on `:8080` is reachable.
- `npm ci` from a committed lockfile; the dependency tree is audited (**0 vulnerabilities** at the time of writing).

Live deployment hardening:

- **JWT verification instead of trust**: the API validates the Cognito ID token's RS256 signature against the pool's JWKS using `node:crypto` (no SDK), then checks `iss`, `aud` / `client_id`, `exp` and `token_use`. A forged, expired, wrong-pool or non-ID token is a `401`
- **Session expiry is enforced client- and server-side**: the guard treats an expired token as signed out, and the API would reject it anyway
- **No SSH on the EC2 host**: port 22 is not opened at all; the security group allows 8080 only from the AWS-managed CloudFront origin-facing prefix list, and administration happens through SSM Session Manager / Run Command
- **IMDSv2 required**, one-hop response limit, and an encrypted `gp3` root volume
- **No long-lived AWS credentials**: GitHub Actions assumes a role via OIDC that is scoped to this repository, this bucket, this distribution and this instance
- **Runtime secrets stay on the host**: the deployed stack's Cognito ids, CORS origin and database passwords are written into a root-only `.env` by the CloudFormation user data script — never in Git, never in an image, and the passwords are `NoEcho` parameters in the stack
- **CloudFront enforces HTTPS** for viewers, and the S3 bucket is private with a bucket policy that only permits that distribution (OAC)

🧪 Testing Checklist

- ☑ Homepage loads successfully. *(smoke check)*
- ☐ Home and Games navigation works.
- ☐ Tournaments and Leaderboard links navigate to their intended sections.
- ☑ Games page loads the catalogue (12 seeded games). *(browser-verified)*
- ☐ Category filters display the expected games.

- ☑ Backend liveness endpoint responds. *(smoke check)*
- ☑ Backend readiness endpoint reports database state. *(smoke check)*
- ☑ Games API returns database records. *(smoke check)*
- ☑ Nginx successfully proxies API requests. *(smoke check on :8080)*
- ☑ Unknown routes return clean JSON 404s. *(smoke check)*
- ☑ Backend port is not reachable from the host. *(verified: connection refused)*
- ☑ Database credentials remain untracked (Git **and** image builds).
- ☑ A clean rebuild + reseed works via `make bootstrap`. *(verified: full teardown → smoke passed)*
- ☑ All 12 catalogue entries open a real game. *(engines tested in CI + browser spot-checks)*
- ☑ Game engines survive hundreds of frames and reach their terminal states. *(21 frontend tests)*
- ☑ Every seeded game maps to an engine (no dead catalogue entries). *(mapping contract test)*
- ☑ Cognito tokens are verified server-side; bad/expired/wrong-pool tokens get 401. *(12 auth tests)*
- ☑ Sign-in gate stays open (with a loud boot warning) when no pool is configured. *(auth test)*
- ☑ Live site deployed to S3 + CloudFront, API reachable through CloudFront `/api/*`. *(verified live)*
- ☑ Deployment is keyless: GitHub Actions assumes a role via OIDC. *(deploy workflow)*
- ☑ Structured JSON access logs reach CloudWatch Logs. *(verified with Logs Insights)*
- ☑ Log-derived metrics produce data points. *(verified with `get-metric-statistics`)*
- ☑ The synthetic check runs on schedule and its result is a metric, not just a log line. *(verified live)*
- ☑ Every alarm has a real state and a notification target. *(verified with `describe-alarms`)*

📚 Skills Demonstrated

- Frontend and backend integration
- REST API development
- Relational database connectivity
- MySQL schema bootstrap and seeding
- Docker image creation
- Multi-container application orchestration
- Docker network communication
- Nginx reverse proxy configuration
- Environment-based configuration
- Healthcheck-driven container startup (ordered dependency waits)
- Liveness vs readiness probe design
- Connection pooling
- Non-root container images, pinned base images, audited lockfiles
- One-command bootstrap + smoke testing
- Automated testing (unit + integration) with coverage gates
- CI/CD pipelines (GitHub Actions): lint → test → build → security scan
- Dependency and image security scanning (Trivy, npm audit, gitleaks)

- GDPR-conscious frontend: self-hosted assets, zero third-party requests
- Canvas-based UI animation and game development (6 engines, game-loop design)
- Testing graphics code without a browser (stubbed canvas, frame-driven tests)
- Canvas-based UI animation with accessibility (reduced-motion) support
- Identity and access: Cognito user pools, ID tokens, JWKS-based JWT verification
- Keyless CI/CD: GitHub Actions OIDC federation and least-privilege deploy roles
- Infrastructure as code: five CloudFormation stacks with cross-stack outputs
- Static hosting and CDN design: private S3 + CloudFront OAC, cache-behaviour split
- Observability: structured JSON logs, log-derived metrics, metric math on a dashboard
- Synthetic (black-box) availability monitoring with alarm semantics that treat silence as failure
- Alert design: 8 actionable alarms, SNS notification path, `TreatMissingData` chosen per signal
- SLOs and error budgets written before the incident, with a per-alarm runbook
- Host hardening: SSM-only administration, IMDSv2, prefix-list-restricted ingress
- Git and GitHub workflows
- Application troubleshooting

🗺️ Roadmap

Phase	Focus	Status
0	Reproducible local foundation (compose, seed, healthchecks, `make bootstrap`, smoke tests)	✅ 2026-10-18
1	Automated tests + GitHub Actions CI (lint, coverage gate, Trivy, gitleaks)	✅ 2026-10-18
2	Game-y UI: live animated background, self-hosted game fonts, arcade styling	✅ 2026-10-18
3	Real playable games — every catalogue entry opens a game	✅ 2026-10-19
4	Auth & live: Cognito (sign-in gate), S3 + CloudFront, EC2 backend, CloudFormation + OIDC CD	✅ 2026-10-19
5	Observability: structured logs, metrics from logs, synthetic check, 8 alarms, dashboard, SLOs, runbook	✅ 2026-10-19
6	Local Kubernetes (kind) + Argo CD GitOps	Planned
7	Security & GDPR hardening (deletion flow, SBOM)	Planned
8	/learn documentation site + PDF export + load-test report	Planned

---

👨‍💻 Author

Cijo Stany John

GameHub is an evolving learning and portfolio project built to practise AWS cloud architecture, backend development, containerization, and DevOps deployment workflows.

Built with curiosity, containers, and a passion for gaming. 🎮

Step 0 — Reproducible Local Foundation ✅ (2026-10-18)

> **Verified by:** `scripts/smoke.sh` (5 checks) · full `make clean` → `make bootstrap` teardown-and-rebuild · browser check (12 DB-backed cards)

What was built

The stack went from **not runnable at all** (dead AWS RDS endpoint, no local database, Docker runtime down) to `git clone && make bootstrap` giving a fully working, seeded application with **zero configuration**.

Piece	Decision	Why not the alternative
MySQL **8.4 container** + `db/init.sql`	Auto-seeds schema + 12 games on first boot of an empty volume	*Cloud DB now* → needs an account, costs money, breaks fresh clones; *no DB* → the original state, catalogue dead
Connection pool (`mysql2/promise`)	One shared pool, `connectionLimit: 10`	*Connection per request* (what existed) → TCP+TLS handshake on every request, exhausts MySQL `max_connections` under load
Liveness vs readiness split	`/api/health` never touches the DB; `/api/ready` does	*Health checks the DB* (what existed) → a database blip makes the orchestrator think the app is dead → crash-looping containers
Healthchecks + ordered startup in compose	`db` healthy → `backend` healthy → `frontend` starts	*`depends_on` without conditions* → app starts before MySQL accepts connections → first queries fail
No published API port	Only nginx `:8080` reaches the host	*Publishing `:3000`* → the API bypasses the proxy layer on every developer machine
Pinned images + `npm ci` + non-root	`node:22-alpine`, `nginx:1.29-alpine`, `mysql:8.4`, uid 1000	*`node:alpine` + `npm install`* → unreproducible builds, root processes

The bugs we hit

1. **Critical npm advisory** (`proxy-addr`, IP spoofing via IPv4-mapped IPv6) surfaced by `npm audit` during the build → fixed with `npm audit fix`, lockfile updated, image rebuilt, 0 vulnerabilities since.
2. **A README claim was false**: it said "AWS RDS integration" while the configured endpoint did not even resolve (NXDOMAIN). The honest fix — a local database plus an honest status table — beat any workaround.

Concepts to understand cold

- Why a **pool** beats per-request connections (and what `connectionLimit` protects)
- **Liveness vs readiness**: which probe answers "should I restart you?" vs "should I send you traffic?" — and why mixing them causes crash loops
- **Layer caching**: dependency layer (`package.json` + lockfile) before source so code edits don't re-download packages
- What `docker-entrypoint-initdb.d` does (and why it only runs on an *empty* volume)

Interview questions this unlocks

1. *"Your service is running but `/api/ready` returns 503 — what's happening and what do you do?"*
2. *"How do you make an application reproducible for a new developer?"*
3. *"Why is your backend port not exposed on the host?"*
4. *"The database container was recreated and your data is gone — why, and how do you prevent it?"* (named volume; `make db-reset` is the *intentional* wipe)

Step 1 — Automated Tests + GitHub Actions CI ✅ (2026-10-18)

> **Verified by:** 14/14 tests green · coverage gate (95% lines / 81% branches / 90% functions, gate at 80) · lint clean · CI badge on `main`

What was built

Layer	Tooling	Scope
Unit suite	`node:test` (built-in — **zero test dependencies**)	Liveness, 404 JSON, CORS allow/deny, header hardening, malformed JSON → 400, **DB-outage behaviour**
Integration suite	`node:test` + real MySQL	Readiness, catalogue shape/ordering/uniqueness, category contract with the UI
Gate	`--test-coverage-lines/branches/functions=80`	CI **fails** under 80% — the number is enforced, not decorative
Lint	ESLint 9 flat config	`eslint .` is a CI step, not advice
CI	GitHub Actions, **3 jobs**	① lint+test+coverage vs a MySQL **service container** ② real-image **stack smoke test** ③ **security**: gitleaks / Trivy / npm audit

Design decisions

- **Integration skips loudly instead of failing** when MySQL is unreachable — a fresh clone with Docker off still gets green unit tests, and the console says exactly why integration was skipped.
- `/api/health` **survives a database outage by test**, not by claim: the suite loads the app pointed at a *closed port* and asserts liveness=200, readiness=503. This is the test that prevents the crash-loop anti-pattern from regressing.
- **No supertest/jest**: `node:test` + built-in `fetch` covers everything here — fewer dependencies is itself a supply-chain decision (npm audit stays 0).
- MySQL published on `127.0.0.1` **only** so host-run integration tests work; CI uses a service container instead of the developer's database.

The bugs we hit

1. **Test process hung forever** after passing — the MySQL pool kept the event loop alive. Fix: `after()` now closes keep-alive sockets [`server.closeAllConnections()`] **and** `await pool.end()`.
2. **Coverage gate failed at 78.6% branches** — the honest fix was *new tests* (DB-error 500 path, TLS config branch, JSON parse failure), not a lower threshold. The gate also exposed a real defect: malformed JSON returned 500; it now returns 400 (client error).

Concepts to understand cold

- **Service containers** in GitHub Actions vs local Docker — same MySQL image, different wiring

- What a **coverage gate** does and why coverage ≠ correctness (we test *behaviour*: CORS refusal, outage semantics, error non-leakage)
- **Test isolation**: `node --test` runs each file in its own process — why that matters when tests mutate `process.env` and `require.cache`

Interview questions this unlocks

1. *"Your CI coverage gate fails at 78% — do you lower the threshold?"*
2. *"How would you test that your app behaves correctly during a database outage?"*
3. *"What's the difference between a unit test and an integration test in your repo, and when does each run?"*
4. *"Name two things your security job would catch before a human reviews the PR."*

Step 2 — Game-y UI: Live Background + Self-Hosted Arcade Fonts ✅ (2026-10-18)

> **Verified by:** browser inspection (0 third-party requests, all three font faces active, canvas layer present) · `bg.js` render logic executed against a stubbed DOM (118 draw calls/frame; zero frames under `prefers-reduced-motion`) · `make smoke` 5/5 · CI full-stack job rebuilds the immutable frontend image

What was built

Layer	Implementation	Why this and not the alternative
Fonts	Press Start 2P (arcade accents) + Chakra Petch (headings/body), **self-hosted woff2** with OFL licenses in-repo	*Google Fonts CDN* (what existed — an `@import` in `style.css`) → every visitor's IP is disclosed to Google on page load; German courts ruled this unlawful without consent (GDPR/§25 TTDSG). Kills Lighthouse third-party requests too
Live background	One `<canvas>`: 3-layer parallax starfield (pixel squares), rising neon pixel particles, rare shooting stars — ~118 draw calls/frame, zero dependencies	*CSS-only gradients* (what existed — static) → user explicitly asked for a moving, "games time" background; *a particle library* → supply-chain weight for 150 lines of canvas we control
Always-moving layers	Grid scrolls upward (CSS `background-position` keyframes), one blurred neon orb drifts (`body::after`, fixed layer)	Cheap GPU-friendly transforms; content stays above (negative z-index layers)
Accessibility	`prefers-reduced-motion`: canvas never created, grid/orb animations off	A moving background must not violate user motion preferences
Arcade controls	Press Start 2P on buttons/filters/logo/badges (kept ≤10px — the font is wide), Chakra Petch 700 on headings	Full-page Press Start 2P would destroy readability and overflow cards

The bugs we hit

1. **The edited CSS never appeared in the browser while `curl` showed it served.** Root cause: `Last-Modified: Sep 16` on the old file gave the browser *heuristic freshness* (10% of file age ≈ 3 days) — it never revalidated. Lesson (and production truth): the frontend ships as an **immutable Docker image**, so any UI change requires `docker-compose build frontend` — bind-mounting for "quick edits" would fork local behaviour away from production.
2. **Canvas "wasn't animating" in tooling.** Root cause: `requestAnimationFrame` is paused in hidden windows. The render loop was verified by executing `bg.js` against a stubbed DOM in Node instead — an executable proof that doesn't depend on a visible desktop.

Concepts to understand cold

- **GDPR and third-party assets**: why self-hosting fonts is a legal requirement in Germany, not a preference — and why `font-display: swap` keeps LCP intact

- **Heuristic caching**: no `Cache-Control` header → browser derives freshness from `Last-Modified` age; the debugging workflow when "my change isn't there"
- **Immutable artifacts**: same image in dev, CI, and prod — changes mean a rebuild, and that's the point
- **Canvas vs CSS** for continuous animation: when `requestAnimationFrame` wins (many moving objects) and when transforms/keyframes win (few large shapes)

Interview questions this unlocks

1. *"A style change works in curl but not in the browser — walk me through how you'd diagnose it."*
2. *"Your Lighthouse audit flags fonts.googleapis.com — is that a performance or a compliance problem in Germany? How do you fix both?"*
3. *"How do you make a continuously animated background accessible and battery-friendly?"*
4. *"Why can't developers just edit files inside the running container?"*

Step 3 — Real Playable Games ✅ (2026-10-19)

> **Verified by:** 40 automated tests (19 backend + 21 frontend) · engine tests run the real game loops in Node · browser spot-checks on 4 engines + the 404 path · catalogue→runner click-through · `make smoke` 5/5 · CI full-stack job

What was built

Every one of the 12 catalogue entries now opens an actual, playable game. No placeholders, no alerts — a `play.html?id=<n>` runner that fetches the title, dispatches it to a real engine, and drives a READY → PLAYING → GAME OVER state machine with HUD, per-title high scores (localStorage) and controls help.

Piece	Implementation	Why not the alternative
6 engines (`engines.js`)	snake · breakout · dodge (survival racing) · memory · reflex · pong — all plain canvas 2D, zero dependencies	*12 bespoke games* → weeks of content work with no new DevOps signal; *one engine themed 12 ways* → dishonest ("every game is snake"); 6 engines × 2 titles is real variety at portfolio scale
Title → engine mapping	`ENGINE_MAP` in `play.js` keyed by game **name** (a uniqueness contract already enforced by a backend integration test)	*An `engine` DB column* → schema migration + `make db-reset` for a frontend concern; the name-uniqueness test already guards the invariant
Coverage contract test	`mapping.test.js` parses the DB seed and `play.js` and fails CI if any seeded title lacks an engine (or vice versa)	*Manual checking* → the promise "every game is playable" would silently rot the day someone adds a seed row
Engine tests	The real engines run in Node against a stubbed canvas/DOM (`vm` + Proxy ctx), driven frame-by-frame	*Browser automation* → a hidden window never fires `requestAnimationFrame`; headless pixels are not behaviour. Frame-driving asserts real terminal paths (snake hits the wall, breakout runs out of lives, pong reaches 5)
`GET /api/games/:id`	Single-title endpoint with 404 for unknown **and** non-numeric ids (no crash, no leak)	*Fetching the whole list* → the runner only needs one title; the endpoint is also the natural REST shape a reviewer looks for

The bugs we hit

1. **The engine tests found a real gameplay stall.** In breakout, a dead-centre paddle hit computes `vx = 0` → the ball bounces vertically forever on a stationary paddle and the game can never end. Fixed with a minimum horizontal component (`speed * 0.25`). A test that couldn't fail this way would have been a decoration.
2. **`413` was missing.** Oversized JSON bodies (body-parser's `entity.too.large`) fell through to the central 500 handler. Now: 413 with a clean body — and the fall-through path itself is covered by a test.
3. **The running container served the old API.** `/api/games/1` returned the 404 handler because the backend container still ran the Phase 0 image; local `curl` against the *new* code hid it. Lesson: images are immutable — a route is not deployed until the image is rebuilt.

Concepts to understand cold

- **Game loop fundamentals**: fixed-step update + render, why `dt` clamping matters, and why input should be queued (snake buffers direction changes so a fast double-tap can't reverse the snake into itself)
- **Testing graphics code without pixels**: stub the context, drive frames, assert *state transitions* (score, lives, game over) — behaviour over pixels
- **Contract tests as product promises**: "every catalogue entry is playable" converted into a failing build when violated
- **REST resource design**: list vs detail, idempotent reads, honest 404s for malformed identifiers

Interview questions this unlocks

1. *"How do you test canvas game logic in CI without a browser?"*
2. *"Your test suite can't fail — how do you know it's actually testing anything?"* (the breakout stall is the answer)
3. *"Someone adds a 13th game to the seed — what stops the catalogue from shipping a dead entry?"*
4. *"Why did `curl` succeed while the browser got a 404?"* (immutable images)

Step 4 — Real Auth + Live on AWS ✅ (2026-10-19)

> **Verified by:** 64 automated tests (31 backend + 33 frontend) · Cognito signing key pair exercised with real RSA signatures · live site served from CloudFront · `/api/*` answering through the CloudFront origin · a confirmed test user signing in and calling `/api/me` with a real ID token

What was built

Players now sign up and sign in before they reach the games, and the whole thing runs on AWS as code — no console clicking, no SSH, no stored access keys.

Piece	Implementation	Why not the alternative
Sign-in gate [`frontend/auth.js`]	Raw Cognito REST calls: `SignUp`, `ConfirmSignUp`, `InitiateAuth` [`USER_PASSWORD_AUTH`]	*Amplify or the AWS SDK* → hundreds of KB of JavaScript for three endpoints, plus a bundler this project intentionally does not have
Token verification [`backend/auth.js`]	RS256 checked against the pool JWKS with `node:crypto`; `iss`, `aud` / `client_id`, `exp`, `token_use` all validated	*`jsonwebtoken` + `jwks-rsa`* → two dependencies to do what `createPublicKey({format:"jwk"})` already does; *trusting the frontend* → not security
Enforcement model	Middleware on `/api/*` except `/api/health` and `/api/ready`, active only when the pool is configured	*Always-on* → local development and CI would break or need fake pools; *client-only guard* → trivially bypassed with `curl`
Frontend hosting [`infra/site.yaml`]	Private S3 bucket + CloudFront with Origin Access Control; `/api/*` behaviour with caching disabled	*Public bucket* → everyone can read and write objects; *caching the API* → stale data and cross-user cache confusion
API host [`infra/backend.yaml`]	One EC2 instance running the **same** `docker-compose.yml` as local development, with a 2 GB swap file	*ECS/Fargate + RDS* → more moving parts and cost for a portfolio demo; *Lambda* → the MySQL container would have to go, which is the opposite of "the deployed thing is the local thing"
Access model	No SSH. Ingress on 8080 only from the AWS-managed CloudFront origin-facing prefix list; admin via SSM; IMDSv2 required	*`0.0.0.0/0` on 22 + a key pair* → the most common way a portfolio EC2 becomes someone else's crypto miner
Deploys [`github-oidc.yaml` + `deploy.yml`]	GitHub OIDC → role scoped to this repo, bucket, distribution and instance; site published by `s3 sync`, API host rolled out by an SSM command	*Long-lived IAM keys in GitHub secrets* → a permanent credential to leak; *rebuilding the host per deploy* → slower, and the DB volume lives there

Things that had to be got right

1. **`AutoVerifiedAttributes` + email-only username.** New pools make MFA mandatory unless you opt out, and users sign in with an email address rather than a username. Both are explicit in the template.

2. **Cognito emits `client_id` on access tokens and `aud` on ID tokens.** The verifier accepts either, then rejects non-ID tokens outright — accepting an access token here would hand out a weaker identity contract.
3. **The probes must never 401.** `/api/health` and `/api/ready` are exempt from the gate; an orchestrator that cannot reach its own health endpoint kills a healthy service.
4. **CloudFront needs a domain name, not an IP, for a custom origin.** The backend stack outputs the instance's public DNS name while the elastic IP keeps it stable.
5. **A 1 GB instance running MySQL wants swap.** The user-data script creates a 2 GB swap file before starting the stack — cheap insurance against the OOM killer during a `docker compose build`.
6. **The artifact carries its config, the repo does not.** `config.js` ships with `authEnabled:false` for local development; `scripts/build-frontend.sh` writes the real pool ids into a git-ignored `build/` directory at deploy time.
7. **An edge invalidation is not a browser invalidation.** After a deploy the browser still ran the previous `script.js` from its week-long cache, which made a fixed landing page look broken. The build now stamps every asset URL with the deploy version (`script.js?v=<sha>`) while HTML stays on a one-minute cache — the standard two-tier answer to "how do users get the new version without a hard refresh?"

Concepts to understand cold

- **OIDC federation**: how a GitHub-issued JWT becomes temporary AWS credentials, and why the `sub` condition is what stops any other repository from assuming your deploy role
- **JWKS and key rotation**: why the verifier fetches public keys instead of a shared secret, and why the result is cached
- **Edge vs origin caching**: why static assets get a week and HTML gets a minute, and why `/api/*` must not be cached at all
- **Origin Access Control**: keeping an S3 bucket private while still serving it through a CDN
- **Blast-radius reduction**: no SSH, SSM-only admin, prefix-list ingress, IMDSv2

Interview questions this unlocks

1. *"How does your API know a request comes from a signed-in user?"* (signature, issuer, audience, expiry — and why client-side checks are not the boundary)
2. *"How do your CI pipelines authenticate to AWS, and what happens if the repo is compromised?"* (no keys; role scoped to repo, bucket, distribution, instance)
3. *"Why is the S3 bucket private, and how does the CDN still read it?"* (OAC + bucket policy condition on the distribution ARN)
4. *"What is your EC2 host's attack surface?"* (no SSH, one port, one prefix list, SSM only, IMDSv2)
5. *"How do you deploy config that differs per environment without committing it?"* (generate at build time into a git-ignored directory)

Step 5 — Observability ✅ (2026-10-19)

> **Verified by:** the CloudWatch agent running on the live host, structured JSON nginx access logs arriving in `/gamehub/prod/nginx-access`, the synthetic canary invoked by hand and by schedule, alarms listed in `OK` / `INSUFFICIENT_DATA` with a real target, and one dashboard (`gamehub-prod`) whose log widgets return rows.

What was built

A deployment you cannot see is a deployment you cannot run. This step makes the live system observable from the outside in and the inside out.

Piece	Implementation	Why not the alternative
Structured access logs (`frontend/nginx.conf`)	`log_format … escape=json` with `status` , `uri` , `request_time` , `upstream_time`	*Parsing the default combined log with regexes in a metric filter* → every rule becomes a cubicle joke; JSON metric filters are a first-class CloudWatch feature
Shipping logs + host metrics (`infra/backend.yaml`)	CloudWatch agent installed by user-data, config file written inline, logs → CloudWatch Logs, `mem` / `disk` / `swap` → the `CWAgent` namespace	*Container Insights / an APM vendor* → an agent you must pay per host for, when four metrics and two log files answer every question this project has
Retention (`observability.yaml`)	Log groups created by template with 7-day retention	*Letting the agent create them* → groups that never expire and quietly bill forever is the classic first-week-of-AWS mistake
Metrics from logs (`AWS::Logs::MetricFilter`)	`Requests` , `ServerErrors` , `RequestTimeSeconds` , `ApiRequestTimeSeconds` derived from the JSON log lines	*Instrumenting the app with an EMF library* → more code in the request path for numbers nginx already knows
External availability check	EventBridge `rate(5 minutes)` → a 20-line Lambda that fetches `/` and `/api/health` and prints one JSON line per check; a metric filter turns that line into `SyntheticSuccess`	*CloudWatch Synthetics* → the managed canary costs about $10/month at a 5-minute interval, more than the whole project; printing a line and filtering it is free
Alarms (`observability.yaml`)	8 alarms: availability (synthetic), 5xx count, API p99 latency, instance status, CPU, CPU credits, memory, disk — all publishing to one SNS topic	*A single "something is wrong" alarm* → an alert nobody can act on is an alert everybody mutes
One dashboard	Availability, requests vs errors, latency percentiles, host pressure, and two live log tables (recent 5xx, backend log tail)	*Per-service dashboards* → this service has one host; the dashboard fits on a screen
Written response (`docs/slo.md` , `docs/runbook.md`)	Objectives with error budgets, measurement, and one runbook section per alarm containing the exact commands	*"Investigate and fix"* → the runbook's value is that it was written before the incident, by someone who was not panicking

Things that had to be got right

1. **`TreatMissingData` is a decision, not a default.** For the host metrics, missing data is `notBreaching` — a stopped agent should not page anyone. For the synthetic check it is `breaching`: no data means the checker itself is broken, and an availability monitor that fails silently is worse than none.
2. **The agent must not create the log groups.** If the agent starts first, the groups exist without retention and CloudFormation's `AWS::Logs::LogGroup` fails with `AlreadyExists` on the next deploy. The template owns the groups; the agent only writes into them.
3. **A metric filter is not a log parser you can debug in place.** Pattern syntax errors produce an alarm that never fires — no warning, no log entry. The check that matters is a **manual canary invocation followed by `get-metric-statistics`**, proving a data point exists, not that the template was accepted.
4. ******Metric* dimensions *must match exactly.** `CWAgent` attaches `InstanceId`, and the disk plugin adds `path`, `device` and `fstype` on top. An alarm whose dimension set is one key short sits in `INSUFFICIENT_DATA` forever. The dimensions in this stack were read back from `list-metrics` on the running host, then written into the template — verified, not guessed.
5. **The synthetic check measures the path a user takes.** It calls the CloudFront URL, not localhost — otherwise it proves the host is alive while the CDN is broken, which is exactly the failure it is meant to catch.
6. **Alarms without a subscriber are half a feature.** The topic is created and exported; subscribing is one `aws sns subscribe` because the account's own contact address cannot be read by an alarm template. Documented in `docs/runbook.md` §8 rather than pretended away.
7. **Latency is measured where the user waits.** `$request_time` includes the upstream Express call; `$upstream_time` alone would hide the time nginx spends. Having both on the log line is what lets the runbook split "slow app" from "slow edge" with one query.

Concepts to understand cold

- **The four golden signals** — latency, traffic, errors, saturation — and why this dashboard is organised as exactly those four plus availability
- **Metric filters and JSON patterns** — how a log line becomes a time series, including extracting a *numeric value* from a field [`$.request_time`]
- **Extended statistics** — why the `p99` alarm exists next to the average, and why an average latency is the metric that hides the outage
- **Error budgets** — the shift from "uptime is good" to "the budget decides what we work on next", and what burning 3 h 39 m of a 30-day window means
- **Pull vs push monitoring** — why the synthetic check lives outside the system it watches, in a different service, in a different failure domain
- **Retention and cost of observability** — log ingestion is the subscription trap of cloud monitoring; retention is an architectural choice

Interview questions this unlocks

1. *"How do you know your service is up right now?"* (external synthetic check on the public URL, every 5 minutes, with `TreatMissingData: breaching` — and what that costs, which is nothing)
2. *"Walk me through an incident: the p99 latency alarm fired. What do you do?"* (runbook §3: split `request_time` vs `upstream_time`, then CPU credits, then MySQL swap)

3. *"How do you get metrics out of a system you did not instrument?"* (structured JSON logs → metric filters, versus adding an SDK or an EMF library)
4. *"Your alarm fired at 03:00. Was it a false positive, and how would you know?"* (dimension sets, `TreatMissingData`, evaluation periods — and the manual canary invocation that proves the wiring)
5. *"What does your error budget buy you, and who decided the target?"* (99.5 % on a single-AZ t3.micro is the honest ceiling; the budget decides whether the next sprint is features or reliability)

Step 8 — /learn Documentation Site + PDF Export ✅ (2026-10-19)

> **Verified by:** `node scripts/build-learn.js` run twice in a row (identical output, no drift) · one page per source document plus a landing page and the print guide · `./scripts/build-pdf.sh` rendering `learn/gamehub-guide.pdf` through headless Chrome · a spot-check that the GitHub-style tables in the step notes render as real tables, not pipe soup

What was built

The step notes in `docs/` and the `README.md` used to live only as Markdown in the repository. They are now a small static site under `learn/`, plus a single printable guide, generated by two dependency-free scripts.

Piece	Implementation	Why not the alternative
Site generator (`scripts/build-learn.js`)	Node 22 CommonJS, built-ins only (`node:fs`, `node:path`) — reads `README.md` + `docs/*.md`, writes `learn/index.html`, one `learn/<slug>.html` per document, `learn/guide.html`, `learn/style.css` and `learn/fonts/`	*A static-site framework* (Eleventy, Jekyll, VitePress) → hundreds of transitive packages and a build toolchain to keep updated for a handful of Markdown files; *rendering Markdown in the browser* → every visitor downloads a parser and the page is empty without JavaScript
Hand-written Markdown renderer (~200 lines)	Headings, paragraphs, bold/italic/inline code, fenced code, links, autolinks, images, ordered/unordered lists, nested lists, blockquotes, task-list checkboxes, horizontal rules and **GitHub-style tables**	*marked / markdown-it* → a runtime dependency (and a supply-chain surface) for rules this project's docs actually use; the docs lean on tables, so table support is the part that had to be right
Escaping	Every block escapes text (`&`, `<`, `>`, `"`) before inline formatting; code spans are extracted first and re-escaped on restore	*Trusting the source* → `README.md` legitimately contains `<https://…>` links and `<canvas>` inside code spans; the renderer has to show them literally without ever emitting raw injected HTML
PDF export (`scripts/build-pdf.sh`)	Runs the generator, then prints `learn/guide.html` with headless Chrome/Chromium (`--print-to-pdf`) into `learn/gamehub-guide.pdf`	*A PDF library* (PDFKit, Puppeteer) → Puppeteer bundles a browser; PDFKit lay out Markdown by hand. The machine already has a browser, so the browser does the page layout
Design (`learn/style.css`)	The app's own arcade language — dark panels, cyan/magenta accents, Chakra Petch + Press Start 2P — with a sidebar listing every document and a PDF download button	*GitHub-flavoured CSS* → the docs are part of this project and should look like it

The generated `learn/` directory is committed, not ignored: it is a build output, but a small, reviewable one, and it means the site can be published without running a generator in the deploy path.

How the guide becomes a PDF

`learn/guide.html` is a second rendering of the same documents, print-optimised rather than screen-optimised: an A4 `@page` rule, a cover page with the project title, a contents list, and each document starting on a fresh page with its `<h1>` as the section break. `scripts/build-pdf.sh` locates the browser (`/Applications/Google Chrome.app/…` on macOS, then `chromium` / `google-chrome` on Linux, overridable with `CHROME_BIN`) and prints that one file. Because the guide reuses the same renderer as the site, the PDF cannot drift from what the site shows.

Things that had to be got right

1. **Tables are the whole point.** The step notes and the README are mostly `| … | … |` tables — what was built, why, and why not the alternative. A renderer without GFM table support would have produced an unreadable site, so the table parser (header, separator, alignment from `:---`) was written and checked first.
2. **Inline code inside emphasis.** `**` GET /api/games/:id `**` must produce a `<strong>` wrapping a `<code>`, not literal asterisks. Code spans are pulled out into placeholders before escaping and put back at the end, so they can never be re-parsed as emphasis.
3. **Fonts stay self-hosted.** The site and the PDF both reference `fonts/press-start-2p-latin-400.woff2` and the Chakra Petch faces relatively, and the generator copies them out of `frontend/fonts/` into `learn/fonts/`. There is no Google Fonts import anywhere — the same GDPR reasoning as Phase 2.
4. **A second run changes nothing.** The generator is deterministic: the same Markdown produces the same bytes, so re-running it never shows up as a spurious diff and CI could gate on it if the docs were generated there.

Concepts to understand cold

- **A hand-written renderer vs a framework**: what you actually own when you write ~200 lines of Markdown parsing, and when that trade stops being worth it
- **Escaping order**: why you extract code spans, escape, then format in a single left-to-right pass (so generated HTML is never re-scanned)
- **Print CSS**: `@page size: A4`, `break-before: page`, and `break-inside: avoid` on table rows — how a web page becomes a document that paginates
- **Static hosting with relative paths**: why every asset is referenced relatively (`style.css`, `fonts/…`, `gamehub-guide.pdf`) so the same files work at `/` locally and under `/learn/` on CloudFront

How `/learn/` is served

The site is plain static files with relative references, so it can be published under the `learn/` prefix of the *same* S3 bucket that serves the app, and the *same* CloudFront distribution serves it at `<distribution>/learn/` — no second bucket, no second CDN, no new origin. CloudFront already knows how to serve objects the bucket does not make public (private bucket + Origin Access Control), so the documentation inherits that.

One honest caveat: `scripts/deploy-frontend.sh` publishes the app to the bucket **root** with `--delete` (and a versioned-asset cache split). A `learn/` tree is therefore a separate, prefix-scoped sync rather than something that script does today — a delete-sync of the root would remove it. The generator emits everything the publish step needs; wiring that sync (and a `/learn/` CloudFront behaviour, if it needs different caching) is the remaining deploy-side job.

Interview questions this unlocks

1. *"You generated documentation from Markdown — why not use a static-site framework?"* (dependency budget vs the rules this project actually needs)
2. *"How do you stop user- or author-supplied Markdown from injecting HTML?"* (escape order and code-span placeholders)
3. *"How did you turn a web page into a paginated A4 PDF?"* (print CSS + a headless browser that already exists on the machine)
4. *"Two independent websites live in one S3 bucket — how do you deploy one without deleting the other?"* (prefix-scoped sync; never `--delete` the root)

Runbook — GameHub (prod)

> One section per alarm. Each one says what the alarm means, how to confirm the cause with a command, and what to do about it. Everything here is done over **SSM** — there is no SSH access to the host by design.

0. Ground rules

```
export AWS_REGION=eu-central-1
export INSTANCE_ID=$(aws ec2 describe-instances --region "$AWS_REGION" \
  --filters Name=tag:Name,Values=gamehub-prod* \
  --query "Reservations[].Instances[].InstanceId" --output text)
```

Run a command on the host (no keys, no SSH — IAM decides who may do this):

```
aws ssm send-command --region "$AWS_REGION" --instance-ids "$INSTANCE_ID" \
  --document-name AWS-RunShellScript \
  --parameters 'commands=["docker compose -f /opt/gamehub/docker-compose.yml ps"]' \
  --query Command.CommandId --output text
```

Check every alarm at once:

```
aws cloudwatch describe-alarms --region "$AWS_REGION" \
  --alarm-name-prefix gamehub-prod \
  --query "MetricAlarms[].[AlarmName,StateValue]" --output table
```

The general shape of the response is always the same: **state → evidence → action**, and only ever through code, never the console.

1. `gamehub-prod-synthetic-check-failed`

Means: the external check could not get a `200` from the site or from `/api/health` within 10 seconds. This is the closest thing to "users cannot use the site".

Confirm:

```
curl -sS -o /dev/null -w '%{http_code} %{time_total}s\n' https://<distribution>/
curl -sS -o /dev/null -w '%{http_code} %{time_total}s\n' https://<distribution>/api/health
aws lambda invoke --region "$AWS_REGION" \
  --function-name gamehub-prod-canary --log-type Tail /dev/stdout | tail -5
```

Act:

Evidence	Likely cause	Action
`/` fails, `/api/health` fine	CloudFront/S3 path	check the distribution status, then re-run the deploy workflow (`workflow_dispatch`)
both fail	the EC2 host is down	go to section 4 (instance status)
`/api/health` fails, `/` fine	the Express container is unhealthy	section 3, then `docker compose up -d` on the host

2. `gamehub-prod-http-5xx`

Means: more than 5 responses with `status >= 500` in 5 minutes behind nginx.

Confirm — the log line is JSON, so this is a query rather than a grep:

```
# CloudWatch Logs Insights, /gamehub/prod/nginx-access
fields ts, uri, status, request_time, remote_addr
| filter status >= 500
| sort ts desc
| limit 20
```

Then, on the host:

```
docker compose -f /opt/gamehub/docker-compose.yml logs --tail=200 backend
```

Act: a 5xx caused by the last deploy → `git revert` on `main` and let the pipeline republish. A 5xx from the database (`ER_...` in the log) → check `docker compose ps db` and the MySQL healthcheck; the app returns 503, not a crash, so the frontend stays up.

3. `gamehub-prod-api-latency-p99`

Means: p99 of `/api/*` above 1.5 s for 10 minutes.

Confirm:

```
# CloudWatch Logs Insights
fields ts, uri, request_time, upstream_time
| filter uri like /^\/api/
| stats pct(request_time, 99) as p99, avg(request_time) as avg, count(*) as n by uri
| sort p99 desc
```

The gap between `request_time` and `upstream_time` tells you who is slow: nginx/TLS-side or Express.

Act:

Cause	Action
One expensive endpoint	fix the query or add an index — the catalogue read and the scores read are the only DB paths
CPU saturation (see `cpu-high`)	the instance is the bottleneck: stop a concurrent `docker compose build` and confirm the credit balance is recovering
MySQL swapping	memory alarm follows; the host has 2 GB swap and 1 GB RAM, so a heavy query can push MySQL into swap

4. `gamehub-prod-instance-status-check-failed`

Means: the hypervisor cannot reach the instance, or the OS is not responding. Everything else is noise until this is clear.

Confirm:

```
aws ec2 describe-instance-status --region "$AWS_REGION" \
  --instance-ids "$INSTANCE_ID" --include-all-instances \
  --query "InstanceStatuses[].[InstanceStatus.Status,SystemStatus.Status]"
aws ec2 get-console-output --region "$AWS_REGION" \
  --instance-id "$INSTANCE_ID" --output text | tail -30
```

Act: a *system* status failure is AWS-side — stop and start the instance (the elastic IP keeps the address stable) and let the user-data boot the stack again. An *instance* status failure with an OOM message in the console output means the stack ran out of memory: the container restart policies bring it back, but the fix is a real one (see `cpu-credit-balance-low`).

5. `gamehub-prod-cpu-high` / `cpu-credit-balance-low`

Means: the burstable instance is out of headroom. `CPUUtilization` above 85 % for 15 minutes, or fewer than 20 CPU credits left.

Confirm:

```
aws cloudwatch get-metric-statistics --region "$AWS_REGION" \
  --namespace AWS/EC2 --metric-name CPUCreditBalance \
  --dimensions Name=InstanceId,Value="$INSTANCE_ID" \
  --start-time "$(date -u -v-1H +%FT%TZ)" --end-time "$(date -u +%FT%TZ)" \
  --period 300 --statistics Average

# who is eating the CPU
aws ssm send-command --region "$AWS_REGION" --instance-ids "$INSTANCE_ID" \
  --document-name AWS-RunShellScript \
  --parameters 'commands=["docker stats --no-stream","uptime"]' \
  --query Command.CommandId --output text
```

Act: if a build is running, wait — a `docker compose build` on 2 vCPU is the expected cause. If it is sustained traffic, that is the signal to move the workload off the burstable class (see the roadmap), not to silence the alarm.

6. `gamehub-prod-memory-high` / `gamehub-prod-disk-high`

Means: memory above 90 % (a 1 GB instance running MySQL, Node and nginx) or the root filesystem above 85 % (Docker images and logs live there).

Confirm:

```
aws ssm send-command --region "$AWS_REGION" --instance-ids "$INSTANCE_ID" \
  --document-name AWS-RunShellScript \
  --parameters 'commands=["free -m","df -h /","du -sh /var/lib/docker /var/log/gamehub"]' \
  --query Command.CommandId --output text
```

Act:

Cause	Action
Memory	the swap file in the user-data is the buffer, not the fix: reduce `innodb_buffer_pool_size` in `db/my.cnf` or move the database off the box
Disk from Docker	`docker system prune -af --volumes` removes old layers (it will NOT touch the named volumes — verify with `docker volume ls` first)
Disk from logs	the agent ships logs with 7-day retention in CloudWatch; the local copies are rotated by Docker's `json-file` driver, so a runaway log is a code smell

7. Rollback

The pipeline is the only way changes reach the host, so the rollback is also a pipeline operation:

```
git revert <bad-sha>          # or: git revert --mainline 1 <merge-sha>
git push origin main
```

`deploy.yml` publishes the site and rolls the host to the reverted commit. The frontend image is built on the host from the checked-out commit, so "the previous version" is exactly the previous commit — no image registry needed. Database changes are the one thing a revert does not undo: the schema is additive (see `backend/schema.sql`), which is why it can be.

8. Escalation and evidence

- **Alarms:** `gamehub-prod-alarms` (SNS). Subscribe with `aws sns subscribe --topic-arn <arn> --protocol email --notification-endpoint <you@example.com>` and confirm the mail.
- **Dashboard:** `gamehub-prod` (CloudWatch) — availability, error count, latency percentiles, host pressure, live log tables.
- **Logs:** `/gamehub/prod/nginx-access` (structured JSON, every request) and `/gamehub/prod/app` (container stdout/stderr, JSON from Express).
- **Audit:** CloudTrail records every `ssm:SendCommand`, so "who restarted the host at 03:00" has an answer.

Service level objectives — GameHub

> These are the numbers the project holds itself to, how each one is measured, and what happens when one is missed. They are deliberately modest: this is a single t3.micro instance in one availability zone, and an SLO that claims more than the architecture can deliver is a lie in a markdown file.

1. Objectives

#	Objective	Target (rolling 30 days)	Measured by	Alarm
SLO-1	**Availability** — the public site answers `200`	≥ 99.5 %	Synthetic check every 5 min from the Lambda canary [`SyntheticSuccess`]	`gamehub-prod-synthetic-check-failed`
SLO-2	**API availability** — `/api/*` answers without a 5xx	≥ 99.5 %	nginx access log → metric filter [`ServerErrors` vs `Requests`]	`gamehub-prod-http-5xx`
SLO-3	**API latency** — p99 of `/api/*` request time	≤ 1.5 s	nginx access log `request_time` → metric filter [`ApiRequestTimeSeconds`]	`gamehub-prod-api-latency-p99`
SLO-4	**Host health** — instance status checks pass	no failing check for 2 min	EC2 `StatusCheckFailed`	`gamehub-prod-instance-status-check-failed`
SLO-5	**Host capacity** — CPU, memory and disk stay off the ceiling	CPU < 85 % · memory < 90 % · disk < 85 %	EC2 metrics + CloudWatch agent [`CWAgent`]	`gamehub-prod-cpu-high`, `gamehub-prod-memory-high`, `gamehub-prod-disk-high`, `gamehub-prod-cpu-credit-balance-low`

Two things are intentionally **not** SLOs:

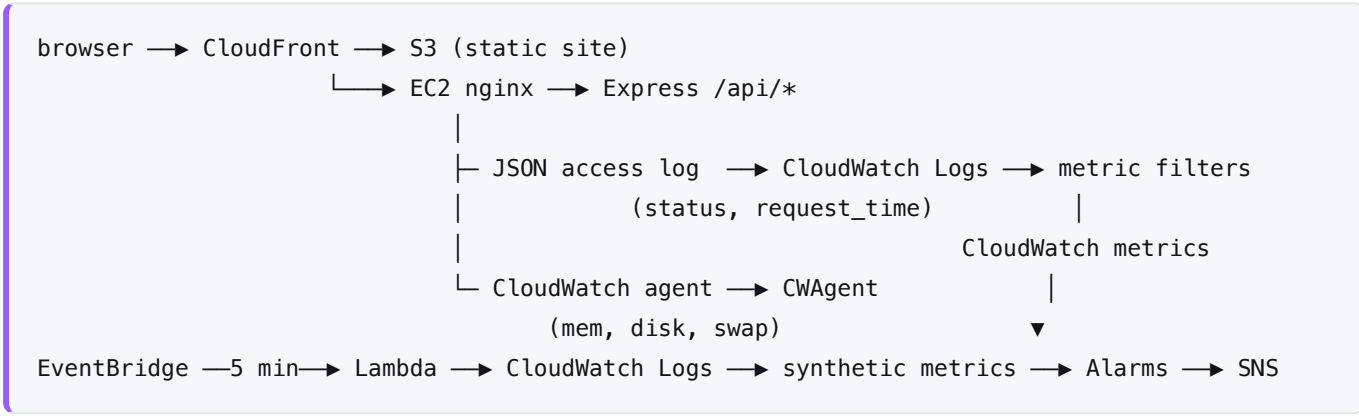
- **Game frame rate** and client-side errors — browser-only metrics, not observable from the server, so claiming a number for them would be theatre.
- **Write durability of scores** — the MySQL volume lives on the instance's EBS root volume. A single-instance topology cannot honestly promise more than "the snapshot schedule in `infra/backend.yaml`".

2. Error budgets

Objective	Budget per 30 days	What "burning" it means
SLO-1 / SLO-2 availability ≥ 99.5 %	3 h 39 m of degraded service	One failed 5-minute synthetic check costs 0.03 % of the budget — 36 consecutive failures exhaust it
SLO-3 latency p99 ≤ 1.5 s	3 h 39 m of p99 above target	The alarm needs 2 consecutive 5-minute windows, so a single slow render is not a page

The budget is not a dashboard number to admire; it decides **priority**. While the budget is intact, new features ship. Once an alarm page is unexplained, the availability work comes first — which for this project means the next roadmap item is postponed, not parallelised.

3. How the numbers are actually produced



- Availability and latency come from **measured traffic and a synthetic check**, never from the alarm configuration.
- `Requests` counts every log line, `ServerErrors` counts `status >= 500`; the error ratio is a CloudWatch **metric math** expression on the dashboard (`ServerErrors / Requests`), not a metric filter, so neither side can drift.
- Latency is the nginx `$request_time`, which includes the upstream Express call — the number a user actually feels, not just the time spent in Node.
- `TreatMissingData` is `breaching` only for the two availability alarms. For the synthetic check, silence means the checker itself died, which is a page.

4. When an objective is missed

1. The alarm fires to the `gamehub-prod-alarms` SNS topic (email subscription).
2. The on-call response is written down in `runbook.md` — one section per alarm, with the exact commands to look at, not advice to "investigate".
3. If the miss was caused by a deploy, the rollback is `git revert` on `main`, which re-triggers the pipeline and republishes the previous commit. The container images are pinned to commits, so the previous version is one revert away.
4. If the miss was not caused by a deploy, the fix is a code or infra change on a branch, reviewed and merged like any other change — no hand-editing in the AWS console.